F# Cheat Sheet

Syntax

- `=` equal, `<>` not equal, `&&` `||` `not`, `%` mod, int `/` truncates
- `let rec` for self-calls. `fun x -> x + 1` lambda
- Strings: `s1 + s2`, `String.length s`
- Read types: `int -> int -> bool` = two ints in, bool out
- Tuple: `(1, "a")` `let (a, _, c) = t` `fst p` `snd p` (pairs only)
- List: `[1; 2; 3]` **semicolons**. `[1, 2]` is one tuple!
- `x :: xs` cons. `a @ b` append.
- `List.length` `List.sum` `List.rev` `List.contains x l`
- `List.filter f l` `List.map f l`
- `rec` present?
- Base is `[]`, not `[_]`?
- Recursing on `t`?
- Every case has a result?
- All needed base cases (fib 0 and 1)?
- Branch types match?

Recursion templates

```
let rec f l =
  match l with
  | [] -> 0
  | h :: t -> h + f t         // recurse on t, not h
```

```
let sum l =
  let rec go acc l =
    match l with
    | [] -> acc
    | h :: t -> go (acc + h) t
  go 0 l
```

Practice functions

```
let rec remove x l = match l with
  | [] -> []
  | h::t when h = x -> remove x t
  | h::t -> h :: remove x t

let rec concat a b = match a with
  | [] -> b
  | h::t -> h :: concat t b

let rec zeroToOne l = match l with
  | [] -> []
  | 0::t -> 1 :: zeroToOne t
  | h::t -> h :: zeroToOne t

let removeDups l =
  let rec go seen l = match l with
    | [] -> List.rev seen
    | h::t when List.contains h seen -> go seen t
    | h::t -> go (h::seen) t
  go [] l

let countAboveAvg l =
  let avg = float (List.sum l) / float (List.length l)
  let rec c l = match l with
    | [] -> 0
    | h::t when float h > avg -> 1 + c t
    | _::t -> c t
  c l

let rec sumEven l = match l with
  | [] -> 0
  | h::t -> if h % 2 = 0 then h + sumEven t else sumEven t
```

Trace format

Non-tail: expand `1 + (2 + (3 + 0))`, then unwind. Tail: table of `acc | list | next call`.